

# Claude Certified Architect — Foundations

## Domain 1 Study Guide

### Agentic Architecture & Orchestration

*Heaviest-weighted domain on the exam blueprint (27%)*

#### Scope

Covers Task Statements 1.1 through 1.7: Agentic Loop Lifecycle, Orchestration Patterns, Context Passing with Structured Metadata, Deterministic Enforcement (Prerequisite Gates), Agent SDK Hooks, Task Decomposition Strategies, and Session State & Resumption.

## Contents

- Task 1.1 — Agentic Loop Lifecycle
- Task 1.2 — Orchestration Patterns (Hub-and-Spoke)
- Task 1.3 — Context Passing with Structured Metadata
- Task 1.4 — Deterministic Enforcement (Prerequisite Gates)
- Task 1.5 — Agent SDK Hooks
- Task 1.6 — Task Decomposition Strategies
- Task 1.7 — Session State and Resumption
- Cross-Cutting Exam Traps — Quick Reference
- Synthesis — One-Line Summary Per Task

## Task 1.1 — Agentic Loop Lifecycle

The four-step lifecycle, executed via the Messages API:

- (a) Send a request to Claude, including full conversation history
- (b) Inspect the `stop_reason` field on the response
- (c) If `stop_reason` is `"tool_use"` — execute the requested tool(s), append results to history, send again
- (d) If `stop_reason` is `"end_turn"` — the agent is finished

### `stop_reason` is the only authoritative signal

| Value                   | Meaning                                                    |
|-------------------------|------------------------------------------------------------|
| <code>"tool_use"</code> | Claude wants to call one or more tools — continue the loop |
| <code>"end_turn"</code> | Claude has finished — exit the loop                        |

*It replaces all natural-language parsing and content-type heuristics for deciding when to stop.*

### Two entries appended to history per tool-use iteration

- Claude's own response (the `tool_use` block) — role: `"assistant"`
- The tool result — a NEW message, role: `"user"`, containing a `tool_result` block tagged with the matching `tool_use_id`

`tool_use_id` matters most when Claude requests multiple tools in a single turn — it correlates each result back to its specific request.

### Three anti-patterns

| Anti-pattern                                                 | Why it's wrong                                                                                                                                        |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>response.content[0].type == "text"</code>              | Claude can return text AND a <code>tool_use</code> block in the same response — text at index 0 does not mean finished                                |
| Arbitrary iteration cap as the primary stop mechanism        | Cuts off genuine work (cap too low) or wastes cost (cap too high / broken exit) — a safety net only, never the real exit condition                    |
| <code>tool_choice: {"type": "any"}</code> for the whole loop | Forces a tool call every turn, including when Claude would naturally finish — makes <code>end_turn</code> unreachable, a self-inflicted infinite loop |

#### Correct default

Leave `tool_choice` unset (defaults to `"auto"`) so `end_turn` stays reachable.

## Task 1.2 — Orchestration Patterns (Hub-and-Spoke)

Structure: all communication flows through one coordinator. Subagents never talk to each other — they only know what the coordinator explicitly puts in their prompt.

### Breadth-first decomposition (two phases)

- Phase 1 — generate broad categories: "at least N, don't stop at N if more exist"
- Phase 2 — validate in a SEPARATE pass: "is anything major missing?" (a dedicated second look beats self-critique in the same breath as generation)

### Isolation principle

Every subagent call must explicitly restate: the specific subtopic, the broader research goal, the expected output format, and any relevant prior findings. Nothing is assumed carried over — even things that seem obviously constant, like the research goal, repeated on every single call.

### Aggregation + refinement

- Build a coverage map: per subtopic, does substantive — not just non-empty — evidence exist?
- Re-delegate ONLY the gaps, not everything again
- The real exit condition is the coverage threshold; a max-iteration cap is the safety net only

#### **Exam trap**

A rigid "divide into exactly N subtopics" prompt is a narrow-decomposition failure one level up — it forces merging or dropping categories to hit the number, rather than letting breadth be discovered.

## Task 1.3 — Context Passing with Structured Metadata

Task (renamed Agent in current Claude Code) is a hard gate. It must be explicitly in `allowedTools` for the coordinator to spawn ANY subagent — having subagent definitions in `options.agents` does not imply permission to invoke them. This is a deliberate, separate grant, not an inference.

### Scoped subagent tool access

| Subagent                | Tool restriction                 |
|-------------------------|----------------------------------|
| Web search agent        | WebSearch only — no file access  |
| Document analysis agent | Read / Grep only — no web access |

This forecloses whole categories of attribution error at the tool level.

### The Finding schema — content vs. metadata

| Field                       | Type / notes                                                                     |
|-----------------------------|----------------------------------------------------------------------------------|
| claim                       | Content — the specific fact or insight                                           |
| source_url                  | Metadata — present only when <code>retrieved_by = web_search_agent</code>        |
| document_name / page_number | Metadata — present only when <code>retrieved_by = document_analysis_agent</code> |
| confidence                  | "high"   "medium"   "low"                                                        |
| retrieved_by                | Disambiguates which optional metadata fields should be populated                 |

### The core exam target: the attribution failure pattern

An unsourced claim in a synthesis agent's output is never the synthesis prompt's fault. It is always traced back to the coordinator stripping metadata before the handoff — typically via a shortcut like `.map(f => f.claim).join(...)`, which silently discards every metadata field. The fix: pass the FULL structured array (e.g. `JSON.stringify(allFindings)`) into the synthesis prompt, never just the claim text.

#### Diagnostic order

Unattributed claim → check what data actually reached the synthesis prompt FIRST. Do not start by rewriting the synthesis agent's wording.

### Parallel spawning vs. `fork_session`

| Mechanism                 | Use case                                                                                                                                                       |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parallel Task calls       | Independent subagents, no shared state — coordinator waits for all to finish. Correct for running web search + document analysis concurrently.                 |
| <code>fork_session</code> | Branches an existing conversation's FULL history into a new session — for continuing from shared prior context, not for running independent work concurrently. |

## Task 1.4 — Deterministic Enforcement (Prerequisite Gates)

Prompt instructions ("always verify before refunding") work most of the time — the exam's reference figure is roughly 92% reliable (about an 8% failure rate). A financial operation needs 100%, deterministic enforcement — a plain if-statement checking session state — because code cannot be talked past the way a model's instructions can.

### Mechanism

A session-level state tracker records whether a prerequisite (e.g. `get_customer` returning `verification_status: "verified"`) has actually occurred. The dependent tool (`process_refund`) checks that state before executing anything, and returns a blocked error if the prerequisite hasn't been met — regardless of what the model "decided" to do.

#### Why `input_schema` cannot do this job

JSON Schema only shapes a single call's fields (types, required-ness) — it has no concept of sequencing across calls ("don't allow this tool unless that other tool already ran and returned X"). The gate has to live in your code's dispatcher, not in the schema.

### Structured handoff protocol — required fields

| Field                             | Purpose                                            |
|-----------------------------------|----------------------------------------------------|
| <code>customer_id</code>          | Identifies who this concerns                       |
| <code>conversation_summary</code> | What happened — the human has NO transcript access |
| <code>root_cause</code>           | Why it happened                                    |
| <code>refund_amount</code>        | The specific number, if applicable                 |
| <code>recommended_action</code>   | What the human should do next                      |

An empty or placeholder field is information permanently lost from the human's perspective.

### Multi-concern requests

A bundled request (e.g. `return + billing dispute + account update` in one message) must be decomposed into distinct items, each investigated — ideally in parallel — with the handoff summary covering ALL of them. Free-text `summary/action` fields let one handoff object hold multiple bundled concerns without dropping any.

## Task 1.5 — Agent SDK Hooks

Both hook types let you write your own check logic — the SDK just guarantees WHEN it runs relative to the tool call.

| Hook                                                                                                                                                                                                                                                                                                  | Runs / sees                                                                                    | Correct use                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>PreToolUse</b>                                                                                                                                                                                                                                                                                     | Before the handler executes — only the proposed call (tool name + input); no result exists yet | Policy / security enforcement — deny before the action happens      |
| <b>PostToolUse</b>                                                                                                                                                                                                                                                                                    | After the handler has already run — the actual tool output (real dates, statuses, amounts)     | Data normalisation, or recording state for a later PreToolUse check |
| <b>Exam trap: using PostToolUse to block an action</b><br>By the time PostToolUse runs, the tool's handler has ALREADY executed — for process_refund, the money has already moved. PostToolUse can only inspect or rewrite the result; it cannot undo the action. Blocking must happen in PreToolUse. |                                                                                                |                                                                     |

### Normalisation (PostToolUse)

When multiple tools return heterogeneous formats (Unix timestamps vs. ISO 8601 vs. DD/MM/YYYY; numeric vs. string vs. single-char status codes), a PostToolUse hook rewrites every result into one consistent schema before the model ever sees it — eliminating misinterpretation risk (e.g. confusing day/month order, or reading status "P" as "processed" instead of "pending").

### Threshold + prerequisite gates via PreToolUse

- Deny process\_refund above \$500 with a permissionDecisionReason
- Deny transfer\_funds until session state shows aml\_check returned a pass

Same deterministic-enforcement logic as Task 1.4, formalised as an SDK hook rather than hand-rolled dispatcher logic.

#### Decision framework

Hooks for 100% requirements (compliance, financial thresholds, prerequisite ordering). Prompts for preferences (style, tone, non-critical guidance) where near-total reliability is acceptable.

## Task 1.6 — Task Decomposition Strategies

### Two patterns

| Pattern                                                                                                                                                                                                                                                                  | Best for                                                                                                               | Limitation                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Fixed sequential pipeline</b>                                                                                                                                                                                                                                         | Structured tasks where steps are known in advance (code review, document extraction, compliance checks)                | Cannot adapt to unexpected findings mid-execution               |
| <b>Dynamic adaptive decomposition</b>                                                                                                                                                                                                                                    | Open-ended investigation where scope is unknown at the start (legacy-codebase exploration, security audits, debugging) | Less predictable; harder to estimate time/cost, harder to debug |
| <b>Exam trap</b><br>Matching the pattern to what "sounds more sophisticated" rather than to the task's actual characteristics. A multi-pass code review (N per-file passes + one integration pass) is a FIXED pipeline — the plan is knowable before any file is opened. |                                                                                                                        |                                                                 |

### Attention dilution

A model allocates attention across everything in a single pass — too many items in one pass means less attention per item. Symptoms: detailed feedback on early items, shallow/missing feedback on later ones; the SAME pattern flagged in one item and approved, unflagged, in another.

### The fix is structural, not a model or prompting fix

- Per-item local passes — each item gets its own dedicated call, full attention budget
- A SEPARATE cross-item integration pass, run once after all local passes, checking specifically for cross-cutting concerns that no single-item pass could ever see

#### **Batching alone is NOT the fix**

Batching (e.g. groups of 5) reduces dilution within a batch but never compares items sitting in different batches — cross-batch issues still go undetected without the integration pass.

### Traps to reject explicitly

- A bigger model / larger context window — dilution is architectural, not a capability problem
- A stronger prompt for equal thoroughness — does not solve attention allocation
- Batching without an integration pass — solves within-batch, not cross-batch

## Task 1.7 — Session State and Resumption

Three options, three distinct jobs

| Option                                 | Use when                                                                                              | Do NOT use when                                                  |
|----------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>--resume &lt;name&gt;</b>           | Prior context is still valid — files haven't changed since last session                               | Files have been modified — old tool results go stale in history  |
| <b>fork_session</b>                    | Divergent exploration from a shared, still-valid baseline (e.g. comparing two refactoring strategies) | To fix stale context — the fork still inherits the stale history |
| <b>Fresh start + summary injection</b> | Tool results are stale (files changed) or context has degraded (too cluttered)                        | Prior context is still fully valid — resume is more efficient    |

### The stale context problem (central concept)

Resuming restores EVERY prior tool result, including file reads from before modifications. The model reasons from that old data alongside anything new, producing contradictions — recommending fixes already applied, or referencing code that no longer exists.

#### The naive fix is insufficient

Resuming and asking the agent to re-read changed files does not remove the OLD tool result already sitting in history — it can still influence reasoning on tangential decisions.

#### The correct fix

Fresh session + structured summary (conclusions only — never raw tool output) + explicit naming of which specific files changed, so the agent does TARGETED re-analysis of only those files. The summary covers everything unchanged.

### Exam traps

- Recommending --resume after files changed — stale tool results remain in history
- Using fork\_session to fix stale context — the fork copies the SAME stale history
- Full re-exploration of the whole codebase when only a few files changed — wasteful
- Confusing fork with resume — fork branches (original untouched), resume appends (same session continues)

## Cross-Cutting Exam Traps — Quick Reference

| Trap                                                                     | Why it's wrong                                                                                       | Task      |
|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------|
| <code>response.content[0].type == "text"</code> as the loop exit check   | Ignores everything past index 0 — a <code>tool_use</code> block can follow text in the same response | 1.1       |
| Iteration cap as the primary stop condition                              | Cuts off genuine work OR wastes cost — a safety net only                                             | 1.1       |
| <code>tool_choice: "any"</code> for the whole loop                       | Makes <code>end_turn</code> unreachable — self-inflicted infinite loop                               | 1.1       |
| Rigid "exactly N subtopics" decomposition prompt                         | Forces merging/dropping categories to hit the number                                                 | 1.2       |
| Stripping metadata before synthesis ( <code>.map(f=&gt;f.claim)</code> ) | The single root cause of unattributed-claim failures                                                 | 1.3       |
| Prompt-only enforcement for financial/compliance actions                 | ~92% reliable is not 100% — a gate must be code                                                      | 1.4 / 1.5 |
| <code>PostToolUse</code> used to block an action                         | The handler already ran — too late to prevent it                                                     | 1.5       |
| Fixed pipeline applied to open-ended investigation                       | Cannot adapt to unexpected findings mid-execution                                                    | 1.6       |
| Batching without a cross-item integration pass                           | Solves within-batch dilution only                                                                    | 1.6       |
| Bigger model / larger context as the fix for dilution                    | Dilution is architectural, not a capability limit                                                    | 1.6       |
| <code>--resume</code> after files changed                                | Stale tool results remain in history                                                                 | 1.7       |
| <code>fork_session</code> to fix stale context                           | Fork copies the same stale history                                                                   | 1.7       |
| Full re-exploration when only a few files changed                        | Wasteful — targeted re-analysis is correct                                                           | 1.7       |

## Synthesis — One-Line Summary Per Task

|                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1.1</b><br>stop reason is authoritative; nothing else decides the loop.                                                               |
| <b>1.2</b><br>Breadth-first decomposition + strict subagent isolation, all through one hub.                                              |
| <b>1.3</b><br>Never let structured metadata become plain text before it reaches synthesis.                                               |
| <b>1.4 / 1.5</b><br>Code enforces what prompts can only request.                                                                         |
| <b>1.6</b><br>Match the decomposition pattern to whether the plan is knowable upfront; fix dilution with multi-pass, not a bigger model. |
| <b>1.7</b><br>Stale history beats good intentions — start fresh and inject only what's still true.                                       |

# Claude Certified Architect — Foundations

## Domain 2 Study Guide

### Tool Design and MCP Integration

*18% weight on the exam blueprint*

#### Scope

Covers Task Statements 2.1 through 2.5: Tool Interface Design, Structured Error Responses, Tool Distribution & Tool Choice, MCP Server Integration, and Built-in Tools. Concept notes only — build exercises to follow separately.

## Contents

- Task 2.1 — Tool Interface Design
- Task 2.2 — Structured Error Responses
- Task 2.3 — Tool Distribution & Tool Choice
- Task 2.4 — MCP Server Integration
- Task 2.5 — Built-in Tools
- Cross-Cutting Domain 2 Exam Traps — Quick Reference

## Task 2.1 — Tool Interface Design

### Tool descriptions are the PRIMARY selection mechanism

Not supplementary metadata. When a model receives tools, it reads descriptions to decide which to call. Minimal descriptions like "Retrieves customer information" give it no way to disambiguate overlapping tools.

### Five elements of a production-grade description

- What the tool does — primary purpose, stated unambiguously
- What inputs it expects — data types, formats, constraints, required vs. optional
- Example queries it handles well
- Edge cases and limitations — what it does NOT do
- Explicit boundaries — when to use THIS tool vs. similar tools

### The misrouting fix — ranked

| Option              | Verdict                                                                             |
|---------------------|-------------------------------------------------------------------------------------|
| Expand descriptions | CORRECT — low effort, high leverage, addresses root cause                           |
| Few-shot examples   | WRONG — adds token overhead without fixing why the model is confused                |
| Routing classifier  | WRONG — over-engineered as a first step; bypasses NLU, adds infrastructure          |
| Tool consolidation  | WRONG as first step — valid long-term, but far more effort than better descriptions |

#### The condition that matters

Descriptions are the fix when the agent has a workable number of tools and simply cannot tell two apart. They are NOT the fix when the toolkit itself is the problem — past roughly 4–5 tools per agent, selection degrades on decision complexity alone, and rewriting 22 descriptions leaves that untouched. Diagnose which one you're looking at before reaching for a remedy (see Task 2.3).

### Tool splitting

Generic tools with broad responsibilities create ambiguity. Split `analyze_document` ("Analyses a document and returns results") into `extract_data_points`, `summarize_content`, and `verify_claim_against_source` — each doing one narrow, clearly described job.

### System prompt interactions

#### Subtle failure mode

Keyword-sensitive system prompt instructions ("always check customer details before proceeding") can silently override even well-written tool descriptions, creating unintended tool associations. After updating descriptions, reread the system prompt for conflicts.

## Task 2.2 — Structured Error Responses

### Two error layers

| Layer                 | What happened                                                                                                    | Who handles it                                                         |
|-----------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Protocol errors       | Malformed request — tool doesn't exist, args fail schema, server fault. No result object, only a JSON-RPC error. | Host application. Model never sees a tool result — no tool ran.        |
| Tool execution errors | Tool found, args valid, tool ran, something went wrong inside (upstream API failed, business rule rejected it).  | Normal result with isError: true; model reads it and decides recovery. |

Everything below — the four categories, isRetryable, recovery guidance — lives entirely in the execution-error layer. A protocol error means the call was built wrong; the fix belongs in the schema or client, never the agent's recovery loop.

### The four error categories

| Category   | isRetryable | Recovery                                                                               |
|------------|-------------|----------------------------------------------------------------------------------------|
| Transient  | true        | Resend the same call after a brief delay (timeout, rate limit, service unavailable)    |
| Validation | false       | Fix the input, then send a corrected call — the agent recovers unaided                 |
| Business   | false       | Do NOT retry — take an alternative path or escalate (e.g. refund exceeds policy limit) |
| Permission | false       | Escalate or retry as a principal with the right access                                 |

#### What isRetryable really signals

It answers one narrow question: will resending this EXACT request work? Only transient gets true. false means "not this call again" — not "stop". Validation is recoverable by the agent alone (fix input, resend). Business and permission are not — a policy limit applies regardless of wording, and a permission error needs a different account, not a better call.

#### Where this table comes from

Exam guide v1.0 states retrievable: false for business errors and never assigns a value to validation. The false shown here for validation is the wider-ecosystem convention (gRPC's INVALID\_ARGUMENT, AWS-style retry metadata) applied to a gap the guide leaves open. If a question turns on the boolean for validation, reason from "will resending this exact call work" and land on false.

### Access failure vs. valid empty result — THE distinction to nail

| Scenario       | isError                                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Access failure | true — tool couldn't reach the data source (timeout, auth failure). Data might exist; tool couldn't check. Agent should consider retry. |

| Scenario           | isError                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------|
| Valid empty result | false — tool successfully queried, found nothing. Query executed correctly; there's no data. Agent should NOT retry. |

### The canonical failure mode

A tool returns an empty array after a customer lookup. The agent retries 3 times, then escalates to a human. Analysis reveals the customer's account simply does not exist. The tool succeeded — but because the response doesn't distinguish "couldn't reach the database" from "reached it and found nothing", the agent treats both as failure worth retrying. Fix: structure responses so a successful empty query looks nothing like a failed query.

## Error propagation in multi-agent systems

- Subagents implement local recovery for transient failures first — retry before bothering the coordinator
- Only propagate errors that cannot be resolved locally
- Include partial results and what was attempted: "I searched 3 of 5 sources successfully. Sources 4 and 5 timed out."

Prevents two anti-patterns: silently suppressing errors (returning empty results as success) and terminating entire workflows on a single failure — both leave the coordinator deciding blind.

## Task 2.3 — Tool Distribution & Tool Choice

### The tool overload problem

18 tools on one agent degrades selection reliability — every additional tool adds decision complexity. Optimal range: 4–5 tools per agent, scoped to that agent's specific role. Relevance matters as much as count: a synthesis agent given web\_search tools will tend to misuse them (running its own searches instead of using results already handed to it).

### Which fix applies

| The tools are...                          | The fix                                 |
|-------------------------------------------|-----------------------------------------|
| Few enough to handle, but two read alike  | Sharpen descriptions (Task 2.1)         |
| Different jobs (query, transform, export) | Split by role, 4–5 tools each           |
| Variations on one job, sharing a shape    | Consolidate into one parameterised tool |
| Doing more than the agent should reach    | Constrain them (least privilege)        |

#### The tension the exam likes

Consolidation reduces HOW MANY tools an agent chooses between (19 transformations → 1 transform\_data with an enum). Constraining reduces WHAT ANY ONE TOOL CAN REACH (fetch\_url → load\_document). Collapsing transformations doesn't hand new powers, so it doesn't undo least privilege. Replacing fetch\_url does the opposite job — and both can be correct in the same system.

#### Not a fix

Moving tools onto a second MCP server. Server boundaries are invisible to the model — a client flattens every connected server's tools into one list. A 22-tool problem split across two servers is still a 22-tool problem.

### tool\_choice — three settings

| Setting                   | Behaviour                           | Use for                                                                                      |
|---------------------------|-------------------------------------|----------------------------------------------------------------------------------------------|
| "auto" (default)          | Model decides: tool call or text    | General operation, conversational flexibility                                                |
| "any"                     | MUST call a tool, model picks which | Guaranteed structured output (e.g. unknown document type across multiple extraction schemas) |
| {type: "tool", name: "X"} | MUST call this specific named tool  | Enforcing mandatory first steps (e.g. extract_metadata before enrichment)                    |

### Scoped cross-role tools

A synthesis agent needing constant simple fact-checks shouldn't route every request through the coordinator — that adds 2–3 round trips and can add 40%+ latency. Fix: give the synthesis agent a scoped verify\_fact tool for the ~85% simple-lookup case; complex verifications (multi-source, real judgement) still route through the coordinator.

### Role-specific scoping in practice

| Agent             | Tools (4–5 each)                                                                     |
|-------------------|--------------------------------------------------------------------------------------|
| Web Search        | search_web, fetch_page, extract_links, save_snippet                                  |
| Document Analysis | extract_metadata, extract_data_points, summarize_content, verify_claim               |
| Synthesis         | compile_report, verify_fact (scoped), format_citation, assess_coverage               |
| Coordinator       | Agent (spawns subagents), review_output, request_revision — no domain-specific tools |

## Task 2.4 — MCP Server Integration

### Scoping hierarchy

| Level   | Location                  | Use for                                                                                         |
|---------|---------------------------|-------------------------------------------------------------------------------------------------|
| Project | .mcp.json (repo root)     | Team-wide servers — Jira, GitHub, internal API connectors. Version-controlled, shared on clone. |
| User    | ~/.claude.json (home dir) | Personal/experimental servers. NOT version-controlled, NOT shared.                              |

#### Exam answer vs. current docs

Exam guide v1.0 describes two levels (project vs. user) — that's the exam-keyed answer. Current docs actually describe THREE scopes (local, project, user), with ~/.claude.json holding both local and user entries. On the exam, answer with the two-level split.

Entry shapes: a remote server declares "type": "http" and a url. A local one declares command and args, speaking over stdio. An entry with a url but no type is a configuration error — Claude Code reads it as stdio, skips it.

All tools from all configured servers (project + user) are discovered at connection time and available simultaneously — no manual activation step.

### Environment variable expansion

```
{
  "env": {
    "GITHUB_TOKEN": "${GITHUB_TOKEN}",
    "DATABASE_URL": "${DATABASE_URL}"
  }
}
```

Keeps credentials out of version control — each developer sets tokens locally, config file only references variable names. \${VAR:-default} form falls back when unset. An unset variable with no default does not stop the rest of the config from loading — Claude Code warns and carries on; don't rely on a missing token failing loudly.

### MCP resources

Expose content catalogues upfront so the agent doesn't need exploratory tool calls to discover what exists — issue summaries, documentation hierarchies, database schemas. Resources are application-controlled: the server lists them, but the client decides when to attach one to context (Claude Code does this via @server:resource mentions).

#### Resources vs. tools

Resources show agents what data is available. Tools let them act on it. Without a schema resource, an agent might call list\_tables then describe\_table for every table — burning calls just to get its bearings.

### Build vs. use community servers

- Use community servers for standard integrations (Jira, GitHub, Slack, Linear, Notion) — tested, maintained, updated
- Build custom only when: team-specific workflows community servers can't handle, custom business logic embedded in the tool layer, or proprietary internal systems with no community server

**Exam default**

"Evaluate community servers first" is always correct for a standard integration. "Build custom" is only correct when the scenario explicitly describes team-specific requirements community servers cannot meet.

## Enhancing MCP tool descriptions

A sparse MCP tool description loses out to a built-in tool's rich description (e.g. Grep) even when the MCP tool is more capable — the model simply has better context about built-ins. Fix: write 3-5 sentence descriptions explaining capabilities, outputs, when to use it, and how it compares to the built-in alternative.

## Task 2.5 — Built-in Tools

### Grep vs. Glob — the core distinction

#### One sentence

Grep finds what is INSIDE files (content). Glob finds files by their NAMES (path).

| Tool        | Searches                     | Use for                                                                                   |
|-------------|------------------------------|-------------------------------------------------------------------------------------------|
| <b>Grep</b> | File CONTENTS for patterns   | Function callers, error messages, import statements, variable assignments                 |
| <b>Glob</b> | File PATHS by naming pattern | Test files, config files, all .ts files in a directory (e.g. <code>**/*.test.tsx</code> ) |

#### Exam trap

Using Glob to find function callers fails outright — Glob matches paths, not contents. Using Grep to find test files by naming pattern technically "works" (searching for "test" in filenames as content) but is the wrong tool — the exam expects Glob.

### Read, Write, and Edit

Edit performs targeted modifications using unique text matching — fast and precise, touches only the specific text identified.

#### When Edit fails

Edit requires unique text matching. If `old_string` appears in multiple places, Edit can't tell which you mean and fails — that's a safety mechanism, not a bug.

### Recovery — exam vs. real work

| Context                                | Correct recovery                                                                                                                                                                                                  |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exam guide v1.0</b>                 | Read + Write — read the full file, write back the complete modified version. Works every time, costs a whole file's worth of tokens for what's usually a one-line change.                                         |
| <b>Current Claude Code / real work</b> | Widen <code>old_string</code> with more surrounding context until it pins one location, OR set <code>replace_all: true</code> if every occurrence should change. Both stay on Edit, cost almost no extra context. |

#### Both orderings agree on step one

Try Edit first, always. Never default to Read + Write for every modification — the exam penalises that as burning context tokens. The disagreement is only about what to do AFTER Edit reports a non-unique match.

Deciding between widen vs. `replace_all`: widen when you want to change only ONE specific occurrence (disambiguate by adding surrounding context that's unique to that location); use `replace_all` when every occurrence should genuinely change the same way.

## Incremental codebase understanding

### The costliest mistake

Reading all files upfront. A 200-file codebase read in full swallows the entire context window, mostly on files unrelated to the task. No other exploration mistake costs more.

- Grep to find entry points — anchors the investigation
- Read to follow imports and trace flows — only files justified by step 1
- Grep again to trace usage — catches wrapper/barrel-file renames the first Grep missed
- Read only what you need — each Read justified by the previous step's discovery

## Tracing usage across wrapper modules

`processOrder` is defined in `orders.ts`. Barrel file `utils/index.ts` re-exports it as `submitOrder`; three of five consumers import `submitOrder` from `utils`. A Grep for `processOrder` finds the definition, the barrel line, and only the two consumers using the original name — it CANNOT find the other three, because the string `processOrder` never appears in their files. Read the barrel, spot the rename, Grep for `submitOrder` — the three turn up.

## The deprecation scenario — Grep, then Glob, then Grep again

- Grep for the function name — finds every file whose CONTENTS reference it, including direct-import tests
- Glob for sibling test files (e.g. `OrderProcessor.ts` → `**/OrderProcessor.test.*`) — catches tests that exercise the function indirectly, by PATH convention
- Grep again for wrapper names — catches tests covering the function transitively through a renamed wrapper

### Not Glob first

Content search for direct references, path matching for adjacent tests, content search again for indirect coverage. The sequence is Grep → Glob → Grep, never starting with Glob.

## Cross-Cutting Domain 2 Exam Traps — Quick Reference

| Trap                                                              | Why it's wrong                                                                          | Task |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------|
| Few-shot examples to fix misrouting from minimal descriptions     | Adds token overhead without fixing why the model is confused                            | 2.1  |
| Routing classifier as first fix for misrouting                    | Over-engineered first step; bypasses NLU, adds infrastructure                           | 2.1  |
| Tool consolidation as first fix for misrouting                    | Valid long-term, but far more effort than expanding descriptions                        | 2.1  |
| Ignoring system prompt wording after fixing descriptions          | Keyword-sensitive instructions can silently override good descriptions                  | 2.1  |
| Retrying an empty result from a successful query                  | "No matches" will produce the same empty result every time — accept it                  | 2.2  |
| Generic error messages with no structured metadata                | Agent can't distinguish transient from business-rule failures                           | 2.2  |
| Treating business errors as retryable                             | Same policy violation applies every time — needs an alternative path                    | 2.2  |
| Marking validation isRetryable: true because the agent can fix it | The BOOLEAN is about resending the exact call, not overall recovery                     | 2.2  |
| Reading isRetryable: false as "abandon the task"                  | Validation self-recovers; business/permission need an alternative path, not abandonment | 2.2  |
| Silently suppressing subagent errors as empty-result success      | Hides failure info from the coordinator, preventing intelligent recovery                | 2.2  |
| Routing all simple verifications through the coordinator          | 2–3 round trips per request, up to 40% latency for the 85% simple case                  | 2.3  |
| tool_choice "auto" when structured output is required             | Model may return text instead of a tool call — use "any" or forced selection            | 2.3  |
| Giving one agent 18 tools                                         | Selection reliability degrades — optimal is 4–5 tools per agent                         | 2.3  |
| Generic fetch_url instead of constrained load_document            | Generic tools enable misuse — violates least privilege                                  | 2.3  |
| Building custom MCP server for a standard integration             | Community servers should be evaluated first — build only for team-specific gaps         | 2.4  |
| Team-wide MCP config in ~/.claude.json                            | User-level is personal, not shared — team servers belong in .mcp.json                   | 2.4  |
| Credentials committed directly in .mcp.json                       | Security risk — use \${VAR} expansion so secrets never enter repo history               | 2.4  |
| Sparse MCP tool descriptions                                      | Model prefers richly-described built-ins (e.g. Grep) over an under-described MCP tool   | 2.4  |
| Using Glob to find function callers                               | Glob matches paths, not contents — it cannot search inside files                        | 2.5  |
| Using Grep to find files by extension/naming pattern              | Technically possible but the wrong tool — Glob is purpose-built for paths               | 2.5  |

| Trap                                              | Why it's wrong                                                                       | Task |
|---------------------------------------------------|--------------------------------------------------------------------------------------|------|
| Reading all source files upfront                  | Context-budget killer — use Grep to find entry points first, then Read incrementally | 2.5  |
| Defaulting to Read + Write for every modification | Edit is faster, less context — try Edit first always                                 | 2.5  |

## One-line summary per task

### 2.1

Descriptions are the primary tool-selection signal — but only fix misrouting when the toolkit is small enough to reason about.

### 2.2

isError separates "tool ran" from "tool didn't"; isRetryable separates "try again" from "something must change first".

### 2.3

4–5 tools per agent, scoped to role — consolidate near-duplicates, constrain what's too powerful, and add scoped tools to kill round-trip latency for the common case.

### 2.4

Project config is shared, user config is personal — use community servers by default, and never let sparse descriptions lose to built-ins.

### 2.5

Grep for contents, Glob for paths, Edit before Read+Write, and never read a codebase all at once.

# Claude Certified Architect — Foundations

## Domain 3 Study Guide

### Claude Code Configuration and Workflows

*Second-heaviest domain on the exam blueprint (20%)*

#### Scope

Covers Task Statements 3.1 through 3.6: CLAUDE.md Hierarchy & Scoping, Custom Commands & Skills, Path-Specific Rules, Plan Mode vs Direct Execution, Iterative Refinement Techniques, and CI/CD Integration. Includes a Hands-On Findings section documenting live-tested gaps between documented and actual Claude Code behaviour.

## Contents

- Task 3.1 — CLAUDE.md Hierarchy, Scoping, Modular Organisation
- Task 3.2 — Custom Slash Commands and Skills
- Task 3.3 — Path-Specific Rules for Conditional Convention Loading
- Task 3.4 — Plan Mode vs Direct Execution
- Task 3.5 — Iterative Refinement Techniques
- Task 3.6 — CI/CD Integration
- Cross-Cutting Domain 3 Exam Traps — Quick Reference
- Hands-On Findings — Documented vs. Tested Behaviour

## Task 3.1 — CLAUDE.md Hierarchy, Scoping, Modular Organisation

### Three levels

| Level     | Location                            | Shared via git?    | Use for                                                         |
|-----------|-------------------------------------|--------------------|-----------------------------------------------------------------|
| User      | ~/.claude/CLAUDE.md                 | No                 | Personal preferences, verbosity, own shortcuts                  |
| Project   | .claude/CLAUDE.md or root CLAUDE.md | Yes                | Team-wide standards — naming, error handling, review checklists |
| Directory | <subdir>/CLAUDE.md                  | Yes (if committed) | Package-specific conventions differing from project root        |

### No precedence — only concatenation

All applicable files load into the same context window; none overrides another. Docs are explicit: "if two rules contradict each other, Claude may pick one arbitrarily." CLAUDE.md is delivered as a user message, not the system prompt — that's the actual mechanical reason it can't guarantee compliance the way settings.json can (settings.json is enforced by the client, before Claude ever sees the request; CLAUDE.md is just content Claude interprets, same category as anything else in context).

#### Rule of thumb

If something must hold with zero exceptions (a blocked tool, a mandatory formatter, a financial threshold), it belongs in settings.json or a hook — never CLAUDE.md, regardless of how firmly it's worded ("always," "never"). The right test isn't the wording — it's the blast radius if the instruction is skipped once. Low-cost style/quality preferences ("never leave debug prints") are fine to leave as CLAUDE.md guidance.

### Loading order (order, not precedence)

- Broadest → most specific: user, then project, then directory-level as you go down the tree
- Within a directory, CLAUDE.local.md loads AFTER CLAUDE.md (last-read, not last-word — doesn't win conflicts)
- Files above/at the working directory load in full at launch
- Files in subdirectories are documented to load on demand, when Claude reads a file in that directory — tested live and found unreliable (see Hands-On Findings)

### @ path imports — eager, not lazy

@./path/to/file.md inlines that file's content the moment CLAUDE.md is read — no @import keyword, just @ + path on its own line. This is an organisational tool, not a context-saving one: splitting 600 lines into six 100-line imported files still delivers all 600 lines to Claude every session. Import paths resolve relative to the file containing the @ line, not the working directory.

#### Confirmed live

File count and token count both rose proportionally when an import was added — this mechanism worked exactly as documented in testing.

The actual context-saving mechanism is different: path-scoped .claude/rules/ files (paths: YAML frontmatter) — these only load when Claude is working on a matching file.

### Diagnosing what loaded

/memory lists CLAUDE.md/CLAUDE.local.md/auto-memory locations and can open them. /context reports what actually loaded this session, under "Memory files." Neither command loads anything — both are read-only diagnostics.

**Exam answer**

Exam guide v1.0 predates the split and keys /memory as the answer — give /memory on the exam; use /context at an actual keyboard.

## What survives /compact

Project-root CLAUDE.md returns intact after compaction — not privileged, just re-read from disk, since it was never part of conversation history to begin with. Nested subdirectory CLAUDE.md and path-scoped .claude/rules/ do NOT auto-return; they reload only the next time Claude reads a matching file.

## The exam's favourite trap: new teammate, inconsistent behaviour

Developer A's Claude Code follows conventions perfectly; Developer B clones the same repo, same branch, and gets inconsistent results. Root cause is always the same: A's conventions live in ~/.claude/CLAUDE.md (user-level, invisible to git) instead of project-level. Fix: move the conventions into .claude/CLAUDE.md. Proof mechanism: git status/git diff on the project shows nothing about a user-level file, because git never had access to the home directory at all — that absence IS the proof.

## Task 3.2 — Custom Slash Commands and Skills

### The core distinction (this task statement's hinge)

CLAUDE.md loads every session, automatically. A skill loads only when invoked — explicitly (/name) or when Claude matches your request to its description. Task-specific procedures belong in skills; always-on universal standards belong in CLAUDE.md/.claude/rules/. Never put one where the other belongs.

### Two file shapes, same result

| Shape               | Location                       | Creates |
|---------------------|--------------------------------|---------|
| Flat file           | .claude/commands/<name>.md     | /<name> |
| Folder + entrypoint | .claude/skills/<name>/SKILL.md | /<name> |

**Trap**  
A flat file dropped directly into .claude/skills/ (e.g. .claude/skills/review.md, no folder, no SKILL.md) does NOT create a command — that location only recognises the folder-with-SKILL.md shape. .claude/skills/ is canonical (supports supporting files, auto-discovery, wins on a name clash); .claude/commands/ still works for backward compatibility.

### Scoping — identical pattern to CLAUDE.md

- .claude/skills/ or .claude/commands/ (in the repo) → project-scoped, shared via git
- ~/.claude/skills/ or ~/.claude/commands/ (home directory) → user-scoped, personal, never shared

#### Confirmed live via mistake-then-fix

A skill built at the wrong path (inside the project's .claude/skills/ instead of the home directory) genuinely would have shipped to every teammate on clone — moving it to ~/.claude/skills/ and re-running git status showed it vanish from git's view entirely, proving the boundary directly.

### SKILL.md frontmatter — three fields, with a built-in exam/reality split

| <pre>--- description: "... context: fork allowed-tools:   - Read   - Grep argument-hint: "... ---</pre> |                                                                                 |                                                                                                                         |
|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Field                                                                                                   | Exam guide v1.0 answer                                                          | Actual current behaviour                                                                                                |
| context: fork                                                                                           | Runs skill in isolated sub-agent; keeps verbose output out of main conversation | Same — confirmed live ("Running in the background as @skillname"; main thread only received a condensed summary)        |
| allowed-tools                                                                                           | Restricts the skill to only those tools                                         | Pre-approves those tools (skips permission prompt); everything else still callable. Real restriction = disallowed-tools |
| argument-hint                                                                                           | Prompts the developer for missing arguments                                     | Just a label shown during autocomplete — never interactive                                                              |

**Exam answer vs. tested behaviour must be held separately**  
Give the exam-guide answer on the test, don't let the observed behaviour overwrite it in memory.

description isn't one of the three exam-tested fields but does the most practical work — it's what Claude matches against user intent for auto-invocation. Omit it and the skill only ever fires on explicit /name typing.

## Personal customisation

Create a personal variant under a different name in `~/.claude/skills/` (e.g. team has `/analyse`, you build `/deep-analyse` personally) — doesn't override or conflict with the team version.

## Task 3.3 — Path-Specific Rules for Conditional Convention Loading

### The problem this solves

Conventions that must apply to a file type scattered across many directories — e.g. test files co-located with source across 50+ folders. Neither existing tool fits:

- Root CLAUDE.md → loads every session regardless of relevance (Terraform rules burning tokens while editing React)
- Directory-level CLAUDE.md → would need 50+ copies of the same convention, one per folder, guaranteed to drift out of sync

### The mechanism

.claude/rules/<name>.md, with YAML frontmatter using a paths field and glob patterns:

```
---
paths: ["**/*.test.ts", "**/*.test.tsx", "**/*.spec.ts"]
---
# Test Conventions
...
```

\*\*/= any folder at any depth; \*.test.tsx = any filename ending that way. One file, one pattern, universal coverage — no per-directory copies. The rule loads into context only when Claude is actively working on a matching file; otherwise it stays invisible, saving tokens.

### Decision table

| Scenario                                                                                                                                                                                                      | Best approach              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Universal team standards, all code                                                                                                                                                                            | Root CLAUDE.md             |
| One specific package directory                                                                                                                                                                                | Directory-level CLAUDE.md  |
| A file type spread across many directories                                                                                                                                                                    | Path-specific rules (glob) |
| Task-specific workflow, invoked occasionally                                                                                                                                                                  | Skills                     |
| <b>Exam favourite scenario</b><br>Test files co-located with source across many directories → answer is always path-specific rules with glob patterns, never directory-level CLAUDE.md, never root CLAUDE.md. |                            |

### Skills vs. rules — don't conflate

Both CAN auto-activate via path matching, but they behave differently: rules stay in context as background guidance the moment a matching file is touched, shaping every edit passively. Skills load as a task-style workflow, triggered by intent-match or explicit invocation, and behave as a distinct invocation unit rather than ambient context. "Automatic, always-on convention loading for a file type" → rules. "An occasional task a developer runs" → skills.

## Task 3.4 — Plan Mode vs Direct Execution

The single sentence this task statement hinges on

### Key concept

"The decision is not about difficulty but about ambiguity."

A difficult bug fix with a clear stack trace (known cause, known location, known fix) is still direct execution — no decision remains, just skilled implementation. A simple-sounding feature with multiple valid architectures is still plan mode — the ambiguity is in which approach, not in how hard any one approach is to build.

### Plan mode — when

- Large-scale changes (monolith → microservices, core abstraction refactor)
- Multiple valid approaches exist and must be evaluated
- Architectural decisions with downstream consequences (service boundaries, API contracts)
- Multi-file modifications needing a consistent strategy (a 45-file library migration)
- Codebase exploration needed before anything can safely change

Entered via switching, not asking in prose: `claude --permission-mode plan`, Shift+Tab until status bar shows plan mode on, or `/plan` prefix on a prompt. Writing "in plan mode" inside the prompt body does nothing. Claude reads and proposes; nothing is written to disk until the plan is approved.

### Direct execution — when

- Well-scoped change: single-file fix, clear stack trace
- The correct approach is already known — no design decision to make
- Limited scope: one function, one file

### The hybrid — plan THEN direct, not plan OR direct

Standard real-world pattern, tested on the exam: plan phase explores dependencies, evaluates approaches, designs the strategy → execute phase applies that exact strategy file-by-file under direct execution. A 30-file logging-library migration: plan identifies all importing files and maps API differences; execute applies the migration pattern consistently.

### Recognise complexity upfront — don't wait for surprises

If the task description already states complexity ("restructure the monolith into microservices"), choose plan mode before touching anything — the complexity is stated, not something that emerges through exploration. Starting in direct execution "just to look" risks writing real changes based on an unevaluated approach; discovering a better approach mid-way means unwinding actual file changes, not just discarding an unwritten plan.

### The Explore subagent

Runs codebase discovery (file listings, dependency graphs, code excerpts) in an isolated sub-agent, returning only a condensed summary to the main conversation — keeps the main context window clean so later implementation responses aren't degraded by exploration noise.

### Confirmed live

A real plan-mode run launched 3 parallel Explore agents + 1 Plan agent; main conversation only ever showed short status lines ("Noted — the directory is just a stub..."), never raw file dumps. git status after the full multi-agent exploration-and-design cycle showed zero changes — the no-disk-writes guarantee held completely, even through an interactive decision point.

## Task 3.5 — Iterative Refinement Techniques

### The technique hierarchy

| Situation                                                        | Technique                      |
|------------------------------------------------------------------|--------------------------------|
| You know the exact target; model interprets prose inconsistently | Concrete input/output examples |
| Complex transformation, many edge cases                          | Test-driven iteration          |
| You don't know the target — unfamiliar domain                    | Interview pattern              |

**Self-check question**  
"Do I already know the right answer and just need Claude to hit it reliably — or do I not actually know what the right answer is yet?" First case → examples. Second case → interview pattern.

More precise prose is never the fix for inconsistent interpretation — it still relies on interpretation. Only concrete examples eliminate the ambiguity itself.

### Test-driven iteration

Write tests first — happy path, edge cases, error cases — then share the test failure output, not prose, as the correction signal. "Expected: raises ValueError, Actual: ZeroDivisionError" leaves no room for interpretation the way a paragraph description would.

### Batch vs. sequential feedback

|                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------|
| <b>Self-check question</b><br>"If I fix issue A, does that change what the correct fix for issue B looks like?" |
|-----------------------------------------------------------------------------------------------------------------|

- Yes → batch (one message, all interacting constraints together) — e.g. error response shape + logging format + client SDK types
- No → sequential (separate messages, wait between) — e.g. naming convention and indentation, unrelated fixes

### A caveat worth keeping, from direct testing

Not every prose description is ambiguous enough to actually demonstrate drift across runs — a narrow, conventional fix (e.g. "add error handling to a divide function") can converge on the same solution reliably even without examples, because the solution space is naturally small. The technique hierarchy assumes genuine ambiguity in the task; tasks with real structurally different valid approaches (pagination style, response envelope shape) show the effect far more reliably than narrow bug fixes do.

## Task 3.6 — CI/CD Integration

### The -p flag — highest-weight single fact in Domain 3 (Sample Question 10)

Claude Code defaults to interactive mode — after answering, it waits for the next keyboard input. A CI runner has no keyboard, so the job hangs forever. `-p / --print` is a genuine mode switch: process the prompt once, print to stdout, exit — the interactive wait loop never starts.

```
claude -p "Analyse this pull request for security issues"
```

#### Fabricated distractors to recognise and reject

CLAUDE\_HEADLESS=true (doesn't exist), --batch (doesn't exist), stdin redirection from /dev/null (doesn't address the mode itself, only what interactive mode would read).

### Structured output for machine parsing

- `--output-format json` — wraps the run in a JSON envelope (result, session ID, cost/usage)
- `--json-schema '<schema>'` — validates final output against a schema (print mode only); the validated data lands in `structured_output` — extract with `jq '.structured_output'`, not the envelope's top level

### Session context isolation for review

The same session that generated code is measurably worse at reviewing it — it retains its own justification reasoning and is biased toward not questioning decisions it already defended to itself. Fix: an independent `claude -p` invocation for review, no shared session context.

```
claude -p "Implement the authentication middleware"      # session A
claude -p "Review the authentication middleware for..." # session B, independent
```

### Incremental review context

Without prior findings in context, every run re-derives everything from scratch, and issues the developer saw and chose not to fix get flagged again on every push — eroding trust until developers stop reading comments. Fix: pass previous findings into the next run's prompt, instruct Claude to report only new or still-unaddressed issues.

### CLAUDE.md in CI

Read identically to interactive mode — this is how CI-invoked runs get project context: testing standards, available fixtures, what counts as critical vs. minor, existing coverage. Without it, CI-invoked test generation produces generic boilerplate.

### Key CLI flags

| Flag                                                              | Effect                                                               |
|-------------------------------------------------------------------|----------------------------------------------------------------------|
| <code>--system-prompt / --system-prompt-file</code>               | Replaces the entire default system prompt                            |
| <code>--append-system-prompt / --append-system-prompt-file</code> | Appends to the default prompt                                        |
| <code>--output-format text json stream-json</code>                | Output shape for <code>-p</code>                                     |
| <code>--max-turns &lt;n&gt;</code>                                | Cap agentic turns, then exit                                         |
| <code>--permission-mode &lt;mode&gt;</code>                       | default, acceptEdits, plan, auto, dontAsk, bypassPermissions, manual |
| <code>--allowedTools / --disallowedTools</code>                   | Pre-approve without prompting / deny rules                           |

| Flag             | Effect                                                                 |
|------------------|------------------------------------------------------------------------|
| --add-dir <path> | Grants file access to a directory — not configuration discovery        |
| --bare           | Skips auto-discovery of hooks/skills/plugins/MCP/auto memory/CLAUDE.md |

**Append vs. replace rule**

Append when Claude should stay a coding assistant that also follows extra rules. Replace when the identity or permission model genuinely differs from Claude Code's default (e.g. a non-coding agent in an unwatched pipeline).

GitHub Actions (beyond the guide, but exam-tested — 4–5 questions reported)

anthropics/claude-code-action@v1 runs Claude Code non-interactively as a workflow step, accepting the same CLI flags via `claude_args`. No prompt → interactive mode, waits for `@claude` mention. With a prompt → automation mode on whatever event fired the workflow. `actions/checkout` is required to put the repo on disk. `id-token: write` is needed for the action's GitHub App auth; credentials come from a repository secret, never a literal in the file.

Parallel sessions with git worktrees (beyond the guide, tested since July)

A worktree is a second working directory for the same repo, own branch, shared history — each Claude Code session in its own worktree means edits never collide. `claude --worktree <name>` creates one under `.claude/worktrees/<name>/`. Shared-file coordination is sequencing, not locking: first session merges, second rebases onto the result before touching it.

Batch API vs. real-time — the decision boundary (Sample Question 11)

Message Batches API: 50% cheaper, up to 24-hour processing, no latency SLA.

| Workflow                                                            | Choice                                                             |
|---------------------------------------------------------------------|--------------------------------------------------------------------|
| Pre-merge checks (blocking)                                         | Real-time — developers are waiting, no SLA means unacceptable risk |
| Overnight tech-debt report / weekly audit / nightly test generation | Batch API — not time-sensitive, real savings                       |

**Rule**

Never use Batch API for anything blocking a merge.

## Cross-Cutting Domain 3 Exam Traps — Quick Reference

| Trap                                                                           | Why it's wrong                                                                 | Task |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|------|
| Assuming CLAUDE.md has precedence ("more specific wins")                       | No precedence chain exists — only concatenation; conflicts resolve arbitrarily | 3.1  |
| Thinking /memory triggers loading                                              | It's diagnostic only — config loads automatically based on location            | 3.1  |
| @ imports as a context-saving technique                                        | Eager inlining — same total tokens delivered regardless of file count          | 3.1  |
| Flat .md file dropped directly in .claude/skills/                              | That location requires folder + SKILL.md; a loose file is not picked up        | 3.2  |
| Team command placed in ~/.claude/                                              | User-scoped paths are personal, never shared via git                           | 3.2  |
| Treating a skill as always-on like CLAUDE.md                                   | Skills load on invocation only, not automatically every session                | 3.2  |
| Not using context: fork for verbose skills                                     | Exploratory output pollutes and degrades the main conversation                 | 3.2  |
| Directory-level CLAUDE.md for a file type across 50+ folders                   | Requires one copy per folder — massive maintenance burden, guaranteed drift    | 3.3  |
| File-type conventions placed in root CLAUDE.md                                 | Loads every session regardless of relevance — wastes tokens                    | 3.3  |
| Defaulting to direct execution for multi-file architectural changes            | Risks costly rework when dependencies surface late — plan first                | 3.4  |
| Plan mode for a single-file fix with a clear stack trace                       | Adds unnecessary overhead when cause/location/fix are all already known        | 3.4  |
| Starting direct execution, switching to plan mode only once complexity emerges | Complexity stated in the task description should trigger plan mode upfront     | 3.4  |
| Refining prose harder when interpretation is already inconsistent              | More precise prose still relies on interpretation — examples eliminate it      | 3.5  |
| Confusing interview pattern with examples technique                            | Different problems: unfamiliar domain vs. known-target-inconsistent-output     | 3.5  |
| CLAUDE_HEADLESS=true / --batch / stdin redirection as the CI-hang fix          | None exist or address the actual issue — only -p is real                       | 3.6  |
| Assuming same-session self-review is as effective as independent review        | Retained generation context biases against questioning own decisions           | 3.6  |
| Batch API for pre-merge (blocking) checks                                      | Up to 24-hour processing, no latency SLA                                       | 3.6  |
| Skipping incremental review context                                            | Re-derives everything from scratch — re-flags dismissed issues                 | 3.6  |

### One-line summary per task

#### 3.1

CLAUDE.md has no enforcement guarantee at any scope — settings.json/hooks are for anything that must hold 100%.

**3.2**

A skill is on-demand; CLAUDE.md is always-on — never swap which one holds which kind of instruction.

**3.3**

Glob-scoped rules exist specifically for "one convention, many directories" — the case neither CLAUDE.md variant handles well.

**3.4**

Ambiguity decides the mode, not difficulty — and complexity stated upfront should be acted on upfront.

**3.5**

Show, don't better-describe, when the model already has a known target to hit.

**3.6**

-p is non-negotiable for CI; independent review sessions and incremental context both exist to fight bias and noise.

## Hands-On Findings — Documented vs. Tested Behaviour

### Important

These were found through direct, live testing in a real Claude Code session, not assumed from the docs. For the exam, give the documented answer regardless of what's below — these are field notes for real-world use, not exam corrections.

| Mechanism                                  | Documented behaviour                                       | What live testing showed                                                                                                                                                                                    |
|--------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Subdirectory CLAUDE.md                     | Loads on demand when Claude reads a file in that directory | Did NOT load from a session rooted at repo root, even after explicit file access. DID load reliably when the session was launched WITH that directory as the working directory (walk-up-the-tree at launch) |
| .claude/rules/*.md with paths: frontmatter | Loads only when editing a matching file                    | Did NOT load after creating and referencing a matching file — consistent with multiple open Claude Code bug reports; some suggest an undocumented globs: field works more reliably                          |
| Project-level CLAUDE.md at launch          | Loads in full at session start                             | Confirmed working exactly as documented                                                                                                                                                                     |
| @ path imports                             | Eager inlining, proportional token cost                    | Confirmed working exactly as documented                                                                                                                                                                     |
| context: fork on a skill                   | Isolated sub-agent, clean summary returned                 | Confirmed working exactly as documented                                                                                                                                                                     |
| Plan mode's no-disk-write guarantee        | Nothing written until plan is approved                     | Confirmed working exactly as documented, including through a multi-agent run                                                                                                                                |
| /plan prefix vs. formal plan-mode state    | Three entry routes described as equivalent                 | Saw /plan produce planning behaviour without formally registering plan-mode state internally — self-recovered, but entry routes may not be perfectly interchangeable                                        |

### Practical implication if this matters outside exam prep

If a real project needs the conditional loading mechanisms (subdirectory CLAUDE.md, path-scoped rules) to work reliably, don't assume they will on the current version — either drop the conditional frontmatter and accept always-on token cost, structure the session's working directory to match what's needed at launch, or try the undocumented globs: alternative some bug reports describe as more reliable.

# Claude Certified Architect — Foundations

## Domain 4 Study Guide

### Prompt Engineering and Structured Output

*20% weight on the exam blueprint*

#### Scope

Covers Task Statements 4.1 through 4.6: System Prompts with Explicit Criteria, Few-Shot Prompting, Structured Output with Tool Use, Validation/Retry/Feedback Loops, Batch Processing Strategies, and Multi-Instance & Multi-Pass Review. Closes with a synthesis section connecting Domain 4 back to Domains 1-3.

## Contents

- Task 4.1 — System Prompts with Explicit Criteria
- Task 4.2 — Few-Shot Prompting
- Task 4.3 — Structured Output with Tool Use
- Task 4.4 — Validation, Retry, and Feedback Loops
- Task 4.5 — Batch Processing Strategies
- Task 4.6 — Multi-Instance and Multi-Pass Review
- Cross-Cutting Domain 4 Exam Traps — Quick Reference
- Synthesis — One-Line Summary Per Task

## Task 4.1 — System Prompts with Explicit Criteria

Vague instructions give the model no actionable decision boundary. "Be conservative," "only report high-confidence findings," "use your best judgement" all sound reasonable — which is exactly why the exam uses them as distractors. "Conservative" means different things in different contexts; "high-confidence" is a threshold the model cannot calibrate against anything concrete.

### The fix: explicit categorical criteria

Define precisely what to flag and what to skip:

```
Flag comments only when claimed behaviour contradicts actual code behaviour.  
Report bugs and security vulnerabilities.  
Skip minor style preferences and local patterns.
```

Concrete categories (report X, skip Y) plus a specific trigger (claimed vs. actual contradiction) — not a vibe.

### The false positive trust problem

A high false-positive rate in ONE category destroys trust in ALL categories — trust isn't category-specific, it bleeds across the whole output. If "documentation mismatch" findings are wrong 40% of the time, developers stop reading "security vulnerability" findings too, even at 98% accuracy.

#### The counterintuitive fix

Temporarily DISABLE the high false-positive category while reworking its prompt. Trust in the categories that already work returns immediately once the noisy category stops firing. Re-enable only once precision improves. You're not abandoning the category — you're putting system-wide trust ahead of category completeness.

### Severity calibration needs code examples, not prose

Prose ("Critical: issues that could cause system failures") forces the model to interpret what "could cause failure" means — inconsistent classification results. Code examples at each severity level remove the ambiguity entirely; the model pattern-matches against an actual instance rather than a definition.

### Why confidence-based filtering fails

"Only report high-confidence findings" sounds like good engineering but LLM self-reported confidence is poorly calibrated — the model is often sure about wrong findings and hesitant about right ones. Confidence scores earn their keep in routing (low-confidence → human review, Task 4.6), never as a substitute for explicit criteria.

#### Hierarchy

Explicit criteria first, confidence-based routing second — never skip the first step.

## Task 4.2 — Few-Shot Prompting

Few-shot examples are the first tool to reach for when output is inconsistent — not more instructions, not confidence thresholds, not temperature adjustments. When a thorough prompt still produces different structures across invocations, the fix is showing the exact format, not describing it more precisely.

### Three triggers

- Detailed instructions produce inconsistent formatting — sometimes a list, sometimes a table, sometimes prose
- Inconsistent judgement on ambiguous cases — variable shadowing flagged "critical" in one file, "minor" in another
- Extraction produces empty/null fields for data that exists — information present but in an unexpected format

### Construction rules

- 2–4 targeted examples — fewer doesn't establish a pattern; more wastes tokens without proportional benefit
- Every example must show REASONING, not just input → output
- Cover the actual failing scenarios — not just more examples of what already works

#### Why reasoning matters

Without reasoning, the model learns only the narrow pattern ("queries mentioning order numbers → lookup\_order"). With reasoning ("the specific identifier makes lookup\_order correct over get\_customer, even though this could be read as a general query"), the model learns the general principle and generalises to genuinely novel cases the examples never showed.

### Side effect: hallucination reduction

Examples showing correct extraction across varied document structures (tables vs. narrative, inline citations vs. bibliographies) teach the model to handle structural variety without inventing data — this matters most for documents with genuinely inconsistent formatting page-to-page.

### Few-shot vs. other techniques — match the problem to the fix

| Problem                              | Correct technique                                   |
|--------------------------------------|-----------------------------------------------------|
| Inconsistent output formatting       | Few-shot examples                                   |
| Malformed JSON output                | tool_use with JSON schemas (4.3)                    |
| Fabricated values for missing fields | Optional/nullable schema fields (4.3)               |
| Wrong tool selection                 | Better tool descriptions first (2.1), then few-shot |
| Model misses info in narrative text  | Few-shot showing narrative extraction               |
| Extraction sum doesn't match total   | Validation-retry loop (4.4)                         |

Few-shot is powerful but not universal — the exam tests whether you route each specific failure mode to its actual fix rather than reaching for examples every time.

## Task 4.3 — Structured Output with Tool Use

### The reliability hierarchy

`tool_use` with JSON schemas eliminates JSON syntax errors entirely (missing brackets, trailing commas, unquoted keys) — the schema constrains the SHAPE of what Claude returns. Prompt-based JSON (asking for JSON in a text response) gives no structural guarantee and periodically produces unparseable output in production.

#### Exam answer

Answer with this two-tier hierarchy even though current docs also describe strict: true and `output_config.format` as newer controls the guide predates.

### `tool_choice` — three modes

(Exam answer: three modes. Current docs add a fourth, `{"type": "none"}`.)

| Mode                                   | Behaviour                           | Use for                                                                     |
|----------------------------------------|-------------------------------------|-----------------------------------------------------------------------------|
| "auto" (default)                       | Model decides: tool call or text    | General operation, conversational flexibility                               |
| "any"                                  | MUST call a tool, model picks which | Guaranteed structured output, unknown document type across multiple schemas |
| <code>{type: "tool", name: "X"}</code> | MUST call this specific tool        | Mandatory first step (e.g. <code>extract_metadata</code> before enrichment) |

`tool_choice` applies per request, not per conversation — after a forced call returns, send the next request with `auto` (or omit the parameter), or the model is obliged to call the same tool again and you loop.

### What `tool_use` does NOT prevent — "where the exam gets sneaky"

Schema validation is purely structural. It does NOT catch:

- Sum discrepancies — line items that don't sum to the stated total
- Field placement errors — a date placed in an amount field (both valid strings, schema satisfied)
- Fabrication — the model invents a value for a required field when the source lacks the information

#### All three pass clean schema validation

The schema guarantees structure, never correctness — semantic validation needs separate logic (Task 4.4).

### Schema design for production

- Optional/nullable fields — the primary defence against fabrication
- "unclear" enum value — prevents forcing a classification when evidence is genuinely ambiguous
- "other" + detail string — captures edge cases predefined categories don't cover
- Format normalisation rules in the prompt — the schema enforces structure; the prompt enforces formatting consistency

## Task 4.4 — Validation, Retry, and Feedback Loops

### Retry-with-error-feedback — three required pieces

- The original document — so the model has the source to re-examine
- The failed extraction — so it can see what it produced
- The specific validation error — so it knows exactly what went wrong ("line items sum to £450 but stated\_total is £500")

A naive retry with no specific error reproduces the same mistake — the model has no guidance for what to fix.

### The retry effectiveness boundary — most aggressively tested concept here

| Retries ARE effective for                                                          | Retries are NOT effective for                                                                                               |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Format mismatches, structural output errors, misplaced values, mathematical errors | Information genuinely absent from the source, data in an external document never provided, knowledge the model doesn't have |

If a document genuinely doesn't contain a department name, no amount of retrying produces a correct value — flag for human review or return null.

### Self-correction schema design

- `calculated_total` vs. `stated_total` — extract both; a mismatch is an automatic discrepancy flag with zero external logic needed
- `conflict_detected` booleans — extract both contradictory values and flag the conflict rather than silently picking one
- `detected_pattern` fields — tracks which construct triggered a finding, enabling systematic dismissal-rate analysis

### Schema syntax errors vs. semantic validation errors

Syntax errors (malformed JSON, wrong types) — eliminated entirely by `tool_use` (Task 4.3). Semantic errors (correct JSON, wrong values — sums that don't add up, dates out of order) — require validation logic outside the schema; this is what retry loops exist for. The overlap between 4.3 and 4.4 is intentional: `tool_use` solves the first category, never the second.

### Pydantic as the validation layer

A Pydantic model does two jobs: parsing enforces structure (types, required fields, enums), validators enforce semantics (cross-field arithmetic, date ordering — rules a JSON schema cannot express). Both failure kinds surface through one `ValidationError` with machine-readable, per-field messages — exactly the "specific error" ingredient the retry loop needs.

#### **`strict: true` and `.parse()`**

These enforce the schema server-side; neither removes the need for your own semantic validators — the platform enforces structure, your validators enforce business rules, and the retry loop consumes whichever one fails.

## Task 4.5 — Batch Processing Strategies

### The facts

- 50% cost savings vs. synchronous
- Up to 24-hour processing window
- No guaranteed latency SLA
- No multi-turn tool calling within a single batch request (client tools)
- custom\_id fields correlate request/response pairs

### The matching rule — most tested concept here

| API                | Use for                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Synchronous</b> | Blocking workflows where someone/something is waiting — pre-merge CI checks, real-time review feedback          |
| <b>Batch</b>       | Latency-tolerant workflows consumed later — overnight tech-debt reports, weekly audits, nightly test generation |

#### Exam favourite

A manager proposes switching EVERYTHING to batch for the cost savings. Correct answer keeps blocking workflows synchronous, moves only latency-tolerant ones to batch.

### SLA cadence math — work the arithmetic, don't just state it

30-hour SLA – 24-hour worst-case processing window = 6 hours of buffer. A 6-hour submission CADENCE, however, leaves zero margin: if you submit right at the start of the buffer and that exact batch hits the full 24-hour worst case, it lands precisely at the deadline with no room for anything to go wrong. Submitting every 4 hours means a fresh batch is always in flight — if one run overshoots or expires, the next one, already underway, still lands inside the deadline.

#### The rule

The cadence needs to be shorter than the buffer itself, not equal to it.

### Batch failure handling — three steps

- Identify failures by custom\_id — treat expired as a failure too, that's what an overrun batch returns
- Resubmit ONLY failures, with modifications — chunk oversized docs, simplify prompts, add few-shot examples. Never resubmit the whole batch
- Refine prompts on a 5–10 document sample set BEFORE full submission — a 90% first-pass rate means 100 retries per 1,000 docs; 60% means 400

### Multi-turn tool calling limitation

Exam-keyed answer: no multi-turn tool calling in a batch request — a step needing tool execution mid-processing must run on the synchronous API.

**Current docs split this further**

Server tools (web search) now run their own agentic loop inside a batch item and can return stop\_reason: "pause\_turn". Client tools — your own code's tools — still cannot complete a loop inside one batch item. On the exam, answer per the guide regardless.

## Task 4.6 — Multi-Instance and Multi-Pass Review

### The self-review limitation

A model reviewing its own output in the SAME session retains the reasoning it used to generate that output — it already "knows" why it chose each approach and tends to confirm rather than challenge those decisions. An independent instance (fresh session, no prior reasoning context) judges the output on what it sees alone.

#### Exam-correct fix

A separate model instance — never "please review carefully" instructions in the same session, never extended thinking within the generating session.

### Multi-pass review architecture

(Same underlying mechanism as Domain 1.6 and Domain 3's code-review exercise.) Large reviews suffer attention dilution in a single pass: inconsistent depth, missed bugs in the middle, contradictory findings.

- Pass 1 — per-file local analysis: each file reviewed individually, full attention per file
- Pass 2 — cross-file integration: a SEPARATE pass checking data flow, API consistency, and contradictions across per-file findings

#### Exam distractor to reject

Switching to a larger-context-window model. The problem is attention QUALITY, not context SIZE — only focused per-file passes ensure consistent depth.

### Confidence-based routing

Model self-reports confidence per finding → high confidence reports directly, low confidence routes to human review. The confidence score is NOT self-reported accuracy — it's the model's own read on its certainty, and it needs calibration against labelled examples.

#### Anti-pattern

Raw, uncalibrated confidence for automated decisions. The same raw number (e.g. 0.65) can mean genuinely different real-world accuracy in different pipelines — calibration is pipeline-specific, not a property of the model in general.

### Full production architecture

Generation (first instance) → per-file review (independent instances) → integration review (separate instance) → confidence routing (low-confidence → human) → calibration loop (labelled sets continuously recalibrate thresholds). More expensive than single-pass — worth it when review quality directly affects production reliability.

## Cross-Cutting Domain 4 Exam Traps — Quick Reference

| Trap                                                           | Why it's wrong                                                                | Task |
|----------------------------------------------------------------|-------------------------------------------------------------------------------|------|
| "Be conservative" / "high-confidence only" as a prompt fix     | No actionable decision boundary — the model can't calibrate a vague threshold | 4.1  |
| Keeping a high-false-positive category active while fixing it  | Trust bleeds across ALL categories, not just the broken one                   | 4.1  |
| Prose severity descriptions instead of code examples           | Forces interpretation — inconsistent classification results                   | 4.1  |
| Adding more instructions when output is already inconsistent   | More prose still relies on interpretation — few-shot eliminates the ambiguity | 4.2  |
| Few-shot examples without reasoning                            | Model learns the narrow literal pattern, not the generalisable principle      | 4.2  |
| Confidence thresholds to fix ambiguous judgement calls         | Poorly calibrated — few-shot examples with reasoning are the direct fix       | 4.2  |
| Believing tool_use prevents ALL extraction errors              | Eliminates syntax errors only — sums, placement, fabrication still occur      | 4.3  |
| Confusing tool_choice "auto" with "any"                        | "auto" allows text instead of a tool call — no structured-output guarantee    | 4.3  |
| Making all schema fields required                              | Pressures the model to fabricate values when source information is absent     | 4.3  |
| Assuming retries always work                                   | Cannot create information genuinely absent from the source                    | 4.4  |
| Retrying without the specific validation error                 | Naïve retries reproduce the same mistake                                      | 4.4  |
| Relying on schema validation alone, no semantic checks         | Schema catches syntax; semantic errors need separate validation logic         | 4.4  |
| Treating Pydantic as redundant once tool_use enforces a schema | Schemas can't express cross-field rules — validators do                       | 4.4  |
| Switching all workflows to batch for the cost savings          | Blocking workflows have no SLA guarantee under batch                          | 4.5  |
| Assuming batch results arrive quickly because they often do    | No latency SLA exists — design around the 24-hour maximum                     | 4.5  |
| Batch API for workflows needing multi-turn tool calling        | No multi-turn tool calling in a batch request (client tools)                  | 4.5  |
| Self-review in the same session as a viable strategy           | Retains reasoning context — less likely to question its own decisions         | 4.6  |
| Single-pass review for large multi-file PRs                    | Attention dilution — inconsistent depth, missed bugs, contradictions          | 4.6  |
| Larger context window to fix attention dilution                | The problem is attention quality, not context size                            | 4.6  |
| Uncalibrated confidence scores for automated routing           | Raw self-reported confidence is poorly calibrated                             | 4.6  |

## Synthesis — One-Line Summary Per Task

### 4.1

Explicit categorical criteria beat vague instructions every time — and a broken category left running poisons trust in the accurate ones sitting right next to it.

### 4.2

Few-shot examples fix inconsistency the model already has a clear target for; the reasoning inside each example is what teaches the principle, not just the pattern.

### 4.3

A schema guarantees shape, never correctness — semantic errors (sums, placement, fabrication) walk straight through clean validation.

### 4.4

Retries only fix what's wrong in the extraction, never what's missing from the source — and the specific error, not a bare retry, is what lets the model self-correct.

### 4.5

Match the API to the wait — blocking workflows stay synchronous no matter the cost savings, and batch cadence must be shorter than the SLA buffer, not equal to it.

### 4.6

Fresh eyes catch what generating eyes defend — independent review, per-file passes, and calibrated (not raw) confidence all fight the same bias from different angles.

## How Domain 4 connects to what came before

This domain is less "new concepts" and more "the same few failure patterns, recombined." Worth holding these links explicitly:

- 4.1's trust-bleed and 4.6's self-review bias are both instances of a system's own output quietly corrupting how later output gets judged
- 4.3 → 4.4 is one continuous pipeline: tool\_use narrows the failure surface to semantic errors, then validation/retry actually catches them
- 4.6's multi-pass architecture is the exact mechanism from Domain 1.6 and Domain 3's code-review exercise, applied here to review quality specifically
- 4.6's confidence calibration is the same principle Domain 4.1 already ruled out using raw for filtering — calibration doesn't rescue raw confidence, it replaces it with something measured
- Domain 5 picks up directly where 4.4's retry boundary and 4.6's confidence routing leave off — provenance, escalation, and human-review allocation are this same reliability question asked at the level of an entire system

# Claude Certified Architect — Foundations

## Domain 5 Study Guide

### Context Management and Reliability

*15% weight on the exam blueprint*

#### Scope

Covers Task Statements 5.1 through 5.6: Context Window Management, Escalation & Ambiguity Resolution, Error Propagation in Multi-Agent Systems, Codebase Exploration & Context Degradation, Human Review & Confidence Calibration, and Information Provenance & Multi-Source Synthesis. Closes with a synthesis section connecting Domain 5 back to earlier domains.

## Contents

- Task 5.1 — Context Window Management
- Task 5.2 — Escalation & Ambiguity Resolution
- Task 5.3 — Error Propagation in Multi-Agent Systems
- Task 5.4 — Codebase Exploration & Context Degradation
- Task 5.5 — Human Review & Confidence Calibration
- Task 5.6 — Information Provenance & Multi-Source Synthesis
- Cross-Cutting Domain 5 Exam Traps — Quick Reference
- Synthesis — One-Line Summary Per Task

## Task 5.1 — Context Window Management

### The progressive summarisation trap

Summarising earlier conversation turns to free token budget systematically destroys transactional data — amounts, dates, order numbers, customer-stated expectations. "I'd like a refund of \$247.83 for order #8891 placed on March 3rd" becomes "Customer wants a refund for a recent order" — the three facts the agent needs to process the refund are gone.

#### **The fix: persistent case facts blocks**

Extract transactional facts into a structured block included in EVERY prompt, outside the summarised history. Never summarised, persists across every turn regardless of what happens to conversation history. For multi-issue sessions, give each issue its own entry to prevent cross-contamination during summarisation.

### The "lost in the middle" effect — a different problem entirely

Models process information at the beginning and end of long inputs reliably; findings buried in the middle may be missed or given less weight. Unlike summarisation, nothing here is destroyed — the information is present, verbatim, just positioned somewhere the model statistically attends to less.

#### **The fix is structural, not prompt-based**

Place key findings summaries at the beginning of aggregated inputs, with explicit section headers throughout. A synthesis agent fed three subagents' output should see a "Key Findings Summary" section first, then detailed results with clear boundaries. Telling the model to "pay attention to everything" does not work — position effects are not fixed by instruction.

### Tool result trimming

An order lookup might return 40+ fields; the customer's refund request needs 5. The other 35 consume tokens in every subsequent turn as history grows. Trim to relevant fields BEFORE results enter conversation history — ideally in a PostToolUse hook or the tool implementation itself. Once verbose data is in context, it stays there for every following turn.

### Full conversation history — the API is stateless

Each request must include the complete conversation history; there is no server-side session state. This creates real tension with context limits — the persistent case facts block resolves it by separating critical facts (never summarised) from narrative flow (safe to summarise).

### Upstream agent optimisation

A research subagent's full reasoning chain and raw content wastes a downstream synthesis agent's limited context budget. Modify upstream agents to return structured data — key facts, citations, relevance scores — instead of verbose prose. Structured outputs also let downstream agents process findings without re-parsing prose.

### Prompt caching (real-work mechanics, not exam-scoped beyond existence)

Mark a stable prefix with a cache\_control breakpoint; the API reuses the processed prefix on the next request at a fraction of input cost. Caching matches from the start of the prompt, prefix by prefix — put constant content (system instructions, tool definitions, reference docs) first, volatile content (the user's latest message) after the breakpoint. Get the order wrong and nothing matches; every call pays full price.

## Task 5.2 — Escalation & Ambiguity Resolution

### Three valid escalation triggers

- Explicit human request — honour IMMEDIATELY, no attempt to resolve first. Absolute rule, no exceptions
- Policy exceptions or GAPS (not violations) — a violation has a documented answer ("no"); a gap means the policy is silent on this specific situation
- Inability to make meaningful progress — but only after a genuine attempt; "I might not be able to handle this" isn't sufficient on its own

### Two unreliable triggers — exam anti-patterns

| Trigger                                | Why it fails                                                                                                                                                                                                               |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sentiment-based escalation</b>      | Frustration doesn't correlate with case complexity. A furious customer with a simple late delivery is easy to resolve; a calm customer asking about a policy gap needs real judgement.                                     |
| <b>Self-reported confidence scores</b> | LLM self-confidence is poorly calibrated — often incorrectly confident on hard cases, unnecessarily uncertain on easy ones. Exactly the failure mode where the agent escalates simple cases while attempting complex ones. |

### The frustration nuance

- Straightforward issue + frustrated customer → acknowledge the frustration, offer the resolution in the SAME response. Do not escalate
- Customer reiterates preference for a human AFTER being offered help → escalate now — they've had the opportunity and declined it
- Customer explicitly says "I want a human" from the start → escalate immediately, no investigation first

### Ambiguous customer matching

When a search returns multiple matches (three "John Smith" records), the agent must ask for additional identifiers — email, phone, order number. It must NOT select the most recent record, the most active record, or apply any heuristic. Selecting wrong risks privacy violations (exposing one customer's data to another) or incorrect actions (refunding the wrong account). The only safe response is disambiguation.

### Explicit escalation criteria in system prompts

The proportionate first response, before infrastructure like classifier models or sentiment analysis: add explicit escalation criteria PLUS few-shot examples showing when to escalate, when to resolve autonomously, and the exact handoff format (customer ID, root cause, recommended action). Prompt optimisation always precedes architectural changes.

## Task 5.3 — Error Propagation in Multi-Agent Systems

This task statement is close to a restatement of Domain 2's structured error responses (2.2), applied at the coordinator level instead of a single MCP tool — the genuinely new material here is the two anti-patterns and why they share one root cause.

### Structured error context — four required elements

- Failure type — transient / validation / business / permission
- What was attempted — the specific query, parameters, target system (not "search failed")
- Partial results gathered before failure — discarding 3 of 5 successfully retrieved sources is wasteful
- Potential alternative approaches — the subagent knows its domain; it can suggest a different database, broader terms, cached results

### The two anti-patterns — same root cause, different costumes

| Anti-pattern                | The mechanism                                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Silent suppression</b>   | Returning empty results marked as success. The coordinator believes the search ran and found nothing — it never retries, never tries alternatives. Invisible: the output looks complete but has undetectable gaps. |
| <b>Workflow termination</b> | Killing the entire pipeline on one subagent's failure — discarding completed work from every other subagent that succeeded.                                                                                        |

#### The shared root cause

Both are the only two options available to a coordinator that receives EITHER a bare exception OR a bare empty-success — nothing in between. Structured error context (the four elements above) is what gives the coordinator a genuine middle ground: assess the damage, retry, try an alternative, or proceed with partial results. Eliminating one anti-pattern and eliminating the other both come from the same single fix.

### Access failure vs. valid empty result

Access failure: the tool could not reach the source (timeout, connection error) — the query never executed, consider retry. Valid empty result: the tool reached the source, executed the query, found no matches — this IS the answer, no retry needed. Conflating the two means either never retrying when you should, or wasting time retrying a query that will always return nothing.

### Coverage annotations

When synthesis combines findings from multiple subagents, note which topic areas are well-supported and which have gaps: "Section on geothermal energy is limited due to unavailable journal access during research." Far better than silently omitting the topic — without annotation, a gap looks like the topic was irrelevant rather than that the source was unavailable.

### Local recovery before propagation

Subagents implement local recovery (retry, fallback sources, degraded responses) for transient failures BEFORE propagating to the coordinator. Only escalate what can't be resolved locally — this reduces coordinator complexity, since it doesn't need to manage retry logic for every possible transient failure across every subagent.

## Task 5.4 — Codebase Exploration & Context Degradation

### Context degradation — NOT the same problem as Domain 1's attention dilution

Attention dilution (Domain 1.6) is a BREADTH-in-one-pass problem — 14 files reviewed simultaneously, attention spread thin across all of them at once. Context degradation is a DEPTH-over-time problem — precise findings from turn 3 get buried under verbose output accumulated by turn 20, in one continuous sequential exploration session. It is NOT a token limit problem — increasing the context window does not fix it.

Symptom: the agent starts saying "this follows the typical repository pattern" instead of "the OrderRepository class at src/repos/order.ts implements Repository<T> with custom caching in findById" — it loses specific references as attention shifts to more recent, more verbose output.

### Scratchpad files — the primary mitigation

The agent writes key findings to a file and reads it for subsequent questions. This persists knowledge OUTSIDE the conversation context, making it immune to degradation. Treat this as a deliberate strategy from the outset — not a rescue move once things have already degraded.

### Subagent delegation — context isolation, not just parallelisation

Delegating specific investigation questions to subagents keeps verbose exploration output out of the main agent's context entirely; each subagent explores verbosely in its own isolated context and returns only a structured summary. The main agent's context stays clean for high-level coordination.

#### **Two different intervention points**

Scratchpad files let information SURVIVE despite a context that's filling with verbose output. Subagent delegation prevents verbose output from EVER ENTERING the main agent's context in the first place. Delegation alone, without scratchpad files, can still leave the coordinator vulnerable to degradation over a long enough session — if the coordinator accumulates many clean subagent summaries across 15+ sequential delegations, those summaries themselves pile up and can still push earlier findings into a degraded zone.

### Summary injection between phases

Before spawning Phase 2 subagents, summarise Phase 1 findings and inject that summary into Phase 2's initial context. Prevents the "cold start" problem where Phase 2 duplicates Phase 1 exploration, and ensures Phase 2 has the architectural understanding to ask the right questions.

### /compact — proactive, not just a last resort

Use /compact proactively during extended sessions to protect context QUALITY, not just quantity — not only when hitting context limits.

### Crash recovery via structured state manifests

Each agent exports current state (explored paths, key findings, current phase, next steps) to a known file location. On resume, the coordinator loads this manifest and injects it into agent prompts — the agent picks up where it left off without repeating earlier exploration.

## Task 5.5 — Human Review & Confidence Calibration

### The aggregate metrics trap — most dangerous misconception here

97% overall accuracy can hide catastrophic failure rates on specific document types. Standard invoices dominate volume, so the aggregate looks excellent while several document types sit at unacceptable accuracy.

| Document Type         | Date Acc. | Amount Acc. | Name Acc. |
|-----------------------|-----------|-------------|-----------|
| Standard invoices     | 99.5%     | 98.2%       | 97.8%     |
| Handwritten receipts  | 60.1%     | 55.3%       | 71.2%     |
| Scanned PDFs          | 72.4%     | 69.8%       | 80.1%     |
| International formats | 45.2%     | 52.1%       | 63.4%     |
| Aggregate             | 97.0%     | 96.1%       | 95.8%     |

#### The rule

Always validate accuracy by document type AND field segment before automating. Never make automation decisions based on aggregate metrics alone — the errors hidden by volume-weighting are often the ones with the highest business impact.

### Stratified random sampling — sample high-confidence too

Low-confidence items already route to human review by design — sampling them isn't the gap. The gap is high-confidence, AUTOMATED items: if the model develops a novel error pattern that happens to produce high confidence (e.g. a new supplier's currency-formatting convention fools it into 0.95+ confidence on a wrong amount), routing has no other mechanism to catch it — the item was never reviewed by a human at all. Only stratified sampling of high-confidence extractions detects this.

### Field-level confidence calibration

Raw model confidence is not calibrated — 0.90 on date fields might mean 94% actual accuracy; 0.90 on amount fields might mean only 82%. Calibration requires labelled validation sets: run the model against known-correct extractions, compare confidence to actual accuracy, build a calibration curve PER field type PER document type. The same raw number means different things in different pipelines — calibration is never a one-time, model-wide exercise.

### Reviewer capacity prioritisation

Route the highest-uncertainty items to reviewers first — low confidence, ambiguous/contradictory sources, historically poor-accuracy document types. Do NOT spread capacity evenly; an even split wastes time on items the model already handles well. Prioritisation should be DYNAMIC — the queue reorders continuously by uncertainty, not chronological order.

## Task 5.6 — Information Provenance & Multi-Source Synthesis

This task statement is close to a restatement of Domain 1.3's context-passing failure (metadata silently stripped during a handoff) — applied here to research synthesis instead of a subagent handoff. Same root cause: a synthesis step naturally compresses "\$495 billion in 2023 (IEA, June 2024)" down to "investment has grown significantly."

### Structured claim-source mappings — five required fields

- Claim — the specific assertion
- Source URL — where it was found
- Document name — the source's title
- Relevant excerpt — the specific supporting passage
- Publication date — when published or data collected

Attribution dies at synthesis (the most common failure point) unless the synthesis agent is **EXPLICITLY** instructed to preserve and merge these mappings through the compression step.

### Conflict handling — never arbitrarily select one value

Two credible sources report different numbers for the same measure. Wrong: pick the more recent one, average them, or defer to the more authoritative publisher. Correct: annotate **BOTH** values with full attribution and let the consumer decide.

### Temporal awareness vs. genuine conflict — three distinct cases

| Scenario                                                       | What it actually is                                                   | Correct output                                                                                                 |
|----------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Different publication dates, same methodology                  | A TREND, not a conflict — growth accelerated over the measured period | State both figures with dates; frame as a trend, not a discrepancy                                             |
| Same publication date, different reporting windows/methodology | A methodological difference, not a contradiction                      | Annotate both with attribution AND the methodological explanation (reporting period, calendar vs. fiscal year) |
| Same date, same dataset, same period, same methodology         | A genuine unexplained conflict — no basis to resolve it               | Annotate both values, flag as unexplained, note that no methodological difference accounts for it              |

**The output rule is identical across all three**

"Annotate both values, let the consumer decide" applies regardless of which of the three you're looking at — what changes is the EXPLANATION attached, not whether you pick a winner. Never pick a winner in any of the three cases.

### Content-appropriate rendering

- Financial data → tables (numbers, comparisons, trends read best tabular)
- News and current events → prose (narrative, cause-and-effect, chronology)
- Technical findings → structured lists (architecture, API specs, config options)

Forcing everything into one uniform format degrades readability regardless of which single format is chosen.

### Completing analysis with conflicts intact

When document analysis hits conflicting values, it should complete its work with the conflict included and explicitly annotated — never resolve it unilaterally. That decision belongs to the coordinator or the consumer, who has context the analysis agent may lack.

## Cross-Cutting Domain 5 Exam Traps — Quick Reference

| Trap                                                               | Why it's wrong                                                                                    | Task |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------|
| Believing progressive summarisation is safe for transactional data | Systematically destroys amounts, dates, identifiers — needs a persistent case facts block instead | 5.1  |
| Assuming "pay attention to everything" fixes lost-in-the-middle    | Position effects are structural — fix is placing key findings first, not a prompt reminder        | 5.1  |
| Keeping full untrimmed tool results "in case they're needed later" | 40+ field lookups exhaust token budget across every subsequent turn                               | 5.1  |
| Selectively truncating conversation history                        | API is stateless — breaks coherence; use case facts blocks and summarisation instead              | 5.1  |
| Sentiment-based escalation                                         | Frustration doesn't correlate with case complexity                                                | 5.2  |
| Self-reported confidence for escalation routing                    | Poorly calibrated — confident on hard cases, hesitant on easy ones                                | 5.2  |
| Investigating before honouring an explicit human request           | Absolute rule — escalate immediately, no exceptions                                               | 5.2  |
| Selecting from ambiguous customer matches by recency/activity      | Risks privacy violations and wrong-account actions — always ask for identifiers                   | 5.2  |
| Silent suppression (empty results marked success)                  | Coordinator believes the search succeeded — never retries, never tries alternatives               | 5.3  |
| Workflow termination on one subagent failure                       | Discards completed work from every other successful subagent                                      | 5.3  |
| Retrying a valid empty result                                      | Query executed successfully and found nothing — this IS the answer                                | 5.3  |
| Increasing context window to fix context degradation               | Not a token limit problem — it's an attention/positioning problem                                 | 5.4  |
| Assuming subagent delegation is only about parallelisation         | Primary benefit is context isolation — keeping verbose output out of the main agent               | 5.4  |
| Restarting a session without saving state first                    | Loses all accumulated knowledge — save to scratchpad/manifest before restarting                   | 5.4  |
| Using /compact only when hitting context limits                    | Should be used proactively to protect quality, not just as a last resort                          | 5.4  |
| Using aggregate accuracy to justify full automation                | Hides catastrophic per-document-type failure rates                                                | 5.5  |
| Only sampling low-confidence extractions                           | High-confidence items are automated and unreviewed — novel error patterns go undetected           | 5.5  |
| Using raw confidence scores without calibration                    | Same score means different real accuracy per field/document type                                  | 5.5  |
| Spreading reviewer capacity evenly                                 | Wastes time on high-confidence items — prioritise by uncertainty, dynamically                     | 5.5  |
| Selecting the most recent source when sources conflict             | Destroys information and presents false certainty — annotate both instead                         | 5.6  |

| Trap                                                                  | Why it's wrong                                                                   | Task |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------------|------|
| Assuming different numbers always mean contradiction                  | Different dates/methodologies often explain differences as trends, not conflicts | 5.6  |
| Letting synthesis paraphrase without preserving claim-source mappings | Attribution dies during summarisation unless explicitly preserved                | 5.6  |
| Rendering all content in one uniform format                           | Financial data, news, and technical findings each need a different format        | 5.6  |

## Synthesis — One-Line Summary Per Task

### 5.1

Case facts blocks stop destruction; findings-first placement stops mispositioning — two different problems, two different fixes, never interchangeable.

### 5.2

Escalate on explicit request, policy gaps, and genuine stuck points only — never on sentiment or self-reported confidence, both of which measure the wrong thing entirely.

### 5.3

Silent suppression and workflow termination are the same missing ingredient wearing two costumes — structured error context is the one fix that closes off both.

### 5.4

Context degradation is depth-over-time, not breadth-in-one-pass — scratchpads preserve what's already there, delegation stops the pile-up before it starts.

### 5.5

97% aggregate accuracy is a number that hides exactly the failures that matter most — sample the confident items too, or a new error pattern ships silently.

### 5.6

Attribution dies at the same compression step every time — never let a synthesis pass pick a winner between two sources when it can annotate both instead.

## How Domain 5 connects to what came before

Domain 5 is largely Domain 1-4's reliability patterns re-applied at the level of an entire production system rather than a single call or agent:

- 5.1's case facts block is 4.4's "don't destroy information you'll need later" principle, applied to conversation history instead of an extraction schema
- 5.3 is close to a direct restatement of Domain 2.2's structured error responses (access failure vs. empty result, four categories) at the multi-agent coordinator level
- 5.4's scratchpad/delegation split mirrors Domain 1.3's isolation principle — keep verbose work out of the context that matters, on purpose, from the start
- 5.5's calibration-is-per-pipeline finding is the same conclusion Domain 4.6 reached about confidence scores — a raw number is never portable across contexts
- 5.6's attribution-dies-at-synthesis failure is functionally identical to Domain 1.3's coordinator-strips-metadata bug — different domain, same single point of failure